

Practice Sheet: Review how indexing changes as we move from Python lists, to pandas objects, to NumPy arrays. Then apply the core NumPy skills used in spatial computing.

Try the questions **without** running the code. Then create a JN.

Python indexing refresher

```
locations = ["Oak", "Pine", "Maple", "Cedar", "Elm"]
```

1. What value is returned by `locations[0]`?

Oak

2. What value is returned by `locations[4]`?

Elm

3. What values are returned by `locations[1:4]`?

Pine, Maple, Cedar

4. Write an indexing expression that returns "Maple".

`locations[2]`

```
points = [[100, 200], [140, 230], [180, 260], [120, 280]]
```

5. What does `points[2]` return?

`[[180, 260]]`

6. What does `points[2][1]` return? Explain what each index selects.

260

The first index gets the grouping `[180, 260]`, while the second index selects within the grouping.

7. Write an expression that returns the x-coordinate of the second point. Assume the coordinates are stored as `[x,y]`.

```
points[1][0]
```

pandas indexing refresher

```
import pandas as pd

df = pd.DataFrame({
    "name": ["Oak", "Pine", "Maple", "Cedar"],
    "ft": [25, 48, 31, 60],
    "zone": ["A", "B", "A", "B"] })
```

8. What type of object is returned by `df["ft"]`?

Integer

9. Write code to select the name and ft columns together.

```
Df[ name = Pine, ft = 48, zone = B]
```

10. What row is returned by `df.iloc[1]`?

"ft"

11. Write code using `iloc` to select the first three rows.

```
df.iloc[0:3]
```

12. Write code to select rows where ft is greater than 30.

```
df[df["ft"] > 30]
```

13. What rows are returned by `df.iloc[1:3]`?

Pine and Maple

Moving to NumPy arrays

```
import numpy as np

points = np.array([
    [100, 200],
    [140, 230],
    [180, 260],
    [120, 280]
])
```

14. What does `points.shape` return, and what do the two numbers mean?

(4, 2)

15. What does `points[0]` return?

`array([100, 200])`

16. What does `points[0, 1]` return?

200

17. What does `points[:, 0]` return? Explain the colon.

`array([100, 140, 180, 120])`

The colon means all rows, giving all the x coordinates and the y coordinate of 0.

18. What does `points[:, 1]` return?

`array([200, 230, 260, 280])`

19. Write NumPy indexing code that returns the second and third rows.

`Points[1:3]`

20. Compare `points[2][1]` with `points[2, 1]`.

Both of these commands return 260:

`points[2][1]` indexes twice, first pulling out row 2 and then indexing into the row.

`Points[2, 1]` is a single lookup which uses a row, column pair.

Vectorized operations

```
shift = np.array([10, 20])
```

21. Without running the code, determine the result of `points + shift`.

`[[110, 220], [150, 250], [190, 280], [130, 300]]`

22. Why does this operation not require a Python for loop?

NumPy applies the operation through elements across a whole array. So the loop still technically happens without Python.

Boolean masks and spatial queries

```
x = points[:, 0]
y = points[:, 1]

inside = (x >= 110) & (x <= 150) & (y >= 220) & (y <= 290)
```

23. Write the Boolean values stored in `inside[:]`.
[False, True, False, True]

24. What does `points[inside]` return?

```
array([[140, 230], [120, 280]])
```

25. In GIS language, what spatial operation is this Boolean mask approximating?

Select by location/clipping using a bounding box.

np.where() and classification

```
elevation = np.array([240, 510, 430, 680]) classes
= np.where(elevation >= 500, 1, 0)
```

26. What values are stored in `classes[:]`?

```
[0, 1, 0, 1]
```

27. In your own words, explain the three arguments in `np.where(condition, value_if_true, value_if_false)`.

- Condition is tested for every element.
- Second argument is the value which is used when the condition is True.
- Third argument is the value which is used when the condition is False.

Distance and pandas connection

```
a = np.array([100, 200]) b
= np.array([140, 230])
distance = np.sqrt((a - b) ** 2).sum())
```

28. What does `a - b` return?

```
array([-40, -30])
```

29. Explain why this calculation is an example of **vectorized arithmetic**. `distance = np.sqrt(((a - b) ** 2).sum())`

Subtraction, squaring, and summing all act on coordinates at the same time.

```
ft_array = df["ft"].to_numpy()
```

30. What type of object is `ft_array`?

integer

31. What happened to the pandas index and column label when the Series was converted to NumPy?

They get dropped because NumPy array keeps only the raw values, so the row index (0-3) and "ft" are gone.

Big picture

32. Complete the table by describing what is being selected.

Structure	Expression	What it selects
Python list	<code>locations[2]</code>	Third indexed item
Nested Python list	<code>points_list[2][1]</code>	Third inner list and its second item
pandas DataFrame	<code>df.iloc[2, 1]</code>	Row position 2, Column position 1
NumPy array	<code>points[2, 1]</code>	Row 2, Column 1
NumPy array	<code>points[:, 0]</code>	Column 0 from every row, which would be all x-values.

33. In one sentence: what is the main indexing idea that stays the same across Python, pandas, and NumPy?

They all have an indexing system based on a zero-based position, and all slicing syntax includes the start but excludes the last.

34. What becomes easier when data are arranged as a 2-D NumPy array?

Whole dataset arithmetic becomes more versatile and easier to handle.